

enabled applications are not normally used in production.

DTrace addresses this problem with dynamic instrumentation. How do you wish you could debug and performance tune your web 2.0 applications? What if you could dynamically insert code into a live running application and collect data at the point of instrumentation? What if you could turn these instrumentations on and off dynamically? What if you could use the same instrumentation at every layer of your application? What if this instrumentation supports popular languages such as C, C++, Java, PHP, Python, Ruby, JavaScript? These capabilities are exactly what DTrace provides for you.

DTrace enables you to observe every layer of your application infrastructure. It enables you to collect data for a few seconds (incurring a smaller performance penalty) and turn off data collection dynamically – without restarting applications. You do not need to put new code into your application to use DTrace. DTrace can observe fully performance tuned code – no `-g` option is needed.

## 5.5 Observing multiple layers

D scripts can span multiple layers. The following script reports how much time you are spending on the different layers in an AMP stack (Apache/MySQL/PHP). The output is %s of time spent in Apache, Java, MySQL, the browser, and the operating system. This script is like a TOP tool for AMP.

```
#!/usr/sbin/dtrace -qs
BEGIN
{
    total=mysqlcnt=httpcnt=phpcnt=javacnt=ffxcnt=oth
ercnt=0;
    printf("%10s %10s %10s %10s %10s %10s\n", "%
MYSQL", "% APACHE", \
    "% FIREFOX", "% PHP", "% Java", "% OTHER");
}
php*:::request-startup
{
    inphp[pid,tid]=1;
}
php*:::request-shutdown
{
    inphp[pid,tid]=0;
```

### {dev.works}

Don't forget to visit the definitive resource for programmers from Digit: dev. Works at [devworks.thinkdigit.com](http://devworks.thinkdigit.com)